

CHAPTER 1 · SUB-CHAPTER 1.3

Collections – Set Implementations

HashSet · LinkedHashSet · TreeSet · EnumSet
NavigableSet · Set Algebra · Thread Safety · ConcurrentSkipListSet
equals/hashCode Contract · Bit Vectors · Advanced Internals

~20 Pages

52+ Q&A

Adv. Questions

Interview Ready

■ Purple [ADV] questions indicate advanced / senior-level depth

Section 1 — Set Interface Contract

1.1 What is a Set?

A **Set** is a Collection that contains no duplicate elements. More formally, a Set contains no pair of elements e1 and e2 such that e1.equals(e2), and at most one null element (in most implementations). The Set interface models the mathematical set abstraction.

1.2 Set Interface Key Methods

```
// java.util.Set<E> extends Collection<E>
boolean add(E e); // adds element; returns false if duplicate (no-op)
boolean remove(Object o); // removes element; returns true if it was present
boolean contains(Object o); // returns true if element is present
int size(); // number of elements
boolean isEmpty(); // true if size() == 0
Iterator<E> iterator(); // traverse elements (order depends on implementation)
void forEach(Consumer<? super E> action); // Java 8+ default method

// Bulk operations
boolean addAll(Collection<? extends E> c); // union
boolean removeAll(Collection<?> c); // difference
boolean retainAll(Collection<?> c); // intersection
boolean containsAll(Collection<?> c); // subset check
void clear(); // remove all elements

// Java 8+ Stream support
stream().filter(...).collect(Collectors.toSet());
```

1.3 Set vs List — Key Differences

Feature	Set	List
Duplicates	Not allowed (enforced by equals)	Allowed
Index access	No (no get(int index))	Yes (get(int index))
Ordering	Depends on implementation	Insertion order preserved
Null elements	At most one (HashSet/LinkedHashSet); none (TreeSet/EnumSet)	Multiple nulls allowed
Primary purpose	Membership testing, uniqueness	Ordered sequence of elements
contains() cost	O(1) HashSet, O(log n) TreeSet	O(n) for ArrayList/LinkedList

1.4 Set.of() Factory (Java 9+)

WARNING: Set.of() creates an IMMUTABLE set. Any attempt to add, remove, or clear throws UnsupportedOperationException. Also, Set.of() does NOT allow null elements — passing null throws NullPointerException.

```
// Java 9+ Set.of() – immutable, no nulls, no duplicates
Set<String> roles = Set.of("ADMIN", "USER", "MANAGER");
// roles.add("GUEST"); // UnsupportedOperationException!
// Set.of("A", "A"); // IllegalArgumentException: duplicate element

// Java 10+ Set.copyOf() – defensive copy, also immutable
Set<String> mutable = new HashSet<>(List.of("a", "b", "c"));
Set<String> immutable = Set.copyOf(mutable); // snapshot, immutable

// Java 9+ Set.of() with no elements
Set<Object> empty = Set.of(); // immutable empty set (NOT Collections.emptySet() – different impl)

// Collectors.toUnmodifiableSet() – Stream terminal
Set<String> result = list.stream()
    .filter(s -> s.startsWith("A"))
    .collect(Collectors.toUnmodifiableSet()); // Java 10+
```

1.5 equals/hashCode Contract — Critical for Set Correctness

RULE: If two objects are equal according to equals(), they MUST have the same hashCode(). Violating this contract causes Set to store 'duplicates' silently — the most common Set bug in interviews.

```
// BROKEN: equals overridden but hashCode not – classic interview bug
class Person {
    String name; int age;
    @Override public boolean equals(Object o) {
        if (!(o instanceof Person)) return false;
        Person p = (Person) o;
        return name.equals(p.name) && age == p.age;
    }
    // hashCode NOT overridden -> uses Object.hashCode() (identity hash)
}

Set<Person> set = new HashSet<>();
set.add(new Person("Alice", 30));
set.add(new Person("Alice", 30)); // DIFFERENT hashCode bucket -> stored as duplicate!
System.out.println(set.size()); // 2 (wrong – should be 1)

// FIXED: always override both
@Override public int hashCode() {
    return Objects.hash(name, age);
}
```

Section 2 — HashSet Deep Dive

2.1 Internal Structure

HashSet is backed by a **HashMap** internally. Every element added to the HashSet is stored as a *key* in the underlying HashMap, with a shared dummy constant PRESENT (new Object()) as the value. This design reuses HashMap's well-tested hashing and collision resolution logic without duplicating code.

```
// Simplified JDK source view
public class HashSet<E> extends AbstractSet<E> implements Set<E> {
    private transient HashMap<E, Object> map;
    private static final Object PRESENT = new Object(); // dummy value

    public HashSet() { map = new HashMap<>(); }
    public HashSet(int initialCapacity, float loadFactor) {
        map = new HashMap<>(initialCapacity, loadFactor);
    }

    public boolean add(E e) {
        return map.put(e, PRESENT) == null; // returns true if key was new
    }

    public boolean remove(Object o) {
        return map.remove(o) == PRESENT;
    }

    public boolean contains(Object o) {
        return map.containsKey(o);
    }

    public int size() { return map.size(); }
    public Iterator<E> iterator() { return map.keySet().iterator(); }
}
```

2.2 HashSet Characteristics

Property	Value
Average add/remove/contains	O(1)
Worst-case (all keys same bucket)	O(n) pre-Java 8; O(log n) after treeification
Ordering	No guarantee (depends on hashCode and bucket position)
Null elements	Exactly one null allowed

Default initial capacity	16
Default load factor	0.75 (resize when 75% full)
Thread safety	NOT thread-safe
Iteration order	No guarantee, may change on resize

2.3 HashSet Full Operations Example

```
import java.util.*;
Set<String> set = new HashSet<>();
// Basic operations
set.add("apple"); // returns true (new element)
set.add("banana");
set.add("cherry");
set.add("apple"); // returns false (duplicate)
System.out.println(set.size()); // 3
// Contains and remove
System.out.println(set.contains("banana")); // true
set.remove("banana");
System.out.println(set.contains("banana")); // false
// Null handling
set.add(null); // allowed – at most one null
set.add(null); // returns false (duplicate null)
System.out.println(set.contains(null)); // true
// Iteration – order not guaranteed
for (String s : set) { System.out.println(s); }
set.forEach(System.out::println); // Java 8+
// Bulk operations
Set<String> other = new HashSet<>(Arrays.asList("cherry","date","elderberry"));
set.addAll(other); // union: adds date and elderberry
set.retainAll(other); // intersection: keeps only {cherry, date, elderberry}
set.removeAll(other); // difference: removes cherry, date, elderberry
// Converting from/to collections
List<String> list = new ArrayList<>(Arrays.asList("a","b","a","c","b"));
Set<String> unique = new HashSet<>(list); // {a, b, c} - removes duplicates
List<String> deduped = new ArrayList<>(unique); // back to list (order not preserved)
// Custom capacity & load factor
Set<String> custom = new HashSet<>(100, 0.9f); // less frequent resizing
```

Section 3 — LinkedHashSet Deep Dive

3.1 Internal Structure

`LinkedHashSet` extends `HashSet` and is backed by a **`LinkedHashMap`**. It maintains a **doubly-linked list running through all entries**, which defines the iteration order. The iteration order is the *insertion order* — the order in which elements were inserted into the set. Reinserting an element that is already in the set does NOT affect its insertion order position.

```
// Simplified – LinkedHashSet simply calls super constructor with accessOrder=false
public class LinkedHashSet<E> extends HashSet<E> implements Set<E> {
    public LinkedHashSet() {
        super(16, .75f, true); // calls protected HashSet constructor
    }
    // The protected HashSet constructor creates a LinkedHashMap instead of HashMap:
    // HashSet(int initialCapacity, float loadFactor, boolean dummy) {
    //     map = new LinkedHashMap<>(initialCapacity, loadFactor);
    // }
```

```

}
// LinkedHashMap.Entry extends HashMap.Node and adds:
// Entry<K,V> before, after; // doubly-linked list pointers
// These pointers maintain insertion order independently of bucket structure

```

3.2 LinkedHashMap Example

```

Set<String> lhs = new LinkedHashMap<>();
lhs.add("banana");
lhs.add("apple");
lhs.add("cherry");
lhs.add("apple"); // duplicate – ignored, position NOT changed
lhs.add("date");
// Iteration preserves insertion order: banana, apple, cherry, date
System.out.println(lhs); // [banana, apple, cherry, date]
// Removing and re-adding changes position
lhs.remove("apple");
lhs.add("apple"); // now apple is at end
System.out.println(lhs); // [banana, cherry, date, apple]
// Converting List to Set preserving first occurrence order
List<String> raw = Arrays.asList("b","a","c","a","b","d");
Set<String> ordered = new LinkedHashMap<>(raw); // [b, a, c, d]
List<String> deduped = new ArrayList<>(ordered); // [b, a, c, d]

```

3.3 LinkedHashMap vs HashSet vs TreeSet — At a Glance

Property	HashSet	LinkedHashSet	TreeSet
Order	None	Insertion order	Sorted (natural/comparator)
add/remove/contains	O(1) avg	O(1) avg	O(log n)
Memory overhead	Low	Medium (linked list ptrs)	Medium (tree nodes)
Null allowed	One null	One null	No null
Best use case	Pure uniqueness	Unique + predictable order	Sorted unique elements

Section 4 — TreeSet Deep Dive

4.1 Internal Structure — Red-Black Tree via TreeMap

TreeSet is backed by a **TreeMap**, which itself is a **Red-Black tree** — a self-balancing binary search tree. This guarantees O(log n) worst-case for all add, remove, and contains operations. Elements must either implement **Comparable** or a **Comparator** must be provided at construction time.

```

// TreeSet requires elements to be Comparable, or provide a Comparator
TreeSet<Integer> nums = new TreeSet<>();
nums.add(5); nums.add(2); nums.add(8); nums.add(1); nums.add(9);
System.out.println(nums); // [1, 2, 5, 8, 9] – always sorted ascending
// Custom Comparator – case-insensitive string set
TreeSet<String> words = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
words.add("Banana"); words.add("apple"); words.add("Cherry");
System.out.println(words); // [apple, Banana, Cherry] – sorted case-insensitively
// Reverse order
TreeSet<Integer> desc = new TreeSet<>(Comparator.reverseOrder());
desc.add(3); desc.add(1); desc.add(7);
System.out.println(desc); // [7, 3, 1]
// TreeSet throws ClassCastException if element not Comparable (at add time)
TreeSet<Object> bad = new TreeSet<>();

```

```
bad.add("hello");
// bad.add(42); // ClassCastException: String cannot be cast to Integer
```

4.2 NavigableSet Methods — TreeSet's Superpower

TreeSet implements NavigableSet which extends SortedSet. NavigableSet adds navigation methods that are extremely useful for range queries, nearest-element lookups, and ordered traversal.

```
TreeSet<Integer> ts = new TreeSet<>(Arrays.asList(1,3,5,7,9,11,13,15));
// SortedSet methods (inherited)
System.out.println(ts.first()); // 1 (smallest)
System.out.println(ts.last()); // 15 (largest)
System.out.println(ts.headSet(8)); // [1,3,5,7] – exclusive of 8
System.out.println(ts.tailSet(8)); // [9,11,13,15] – inclusive of 8 (not present, so >=8: 9)
System.out.println(ts.subSet(3, 10)); // [3,5,7,9] – fromInclusive, toExclusive

// NavigableSet methods (TreeSet-specific)
System.out.println(ts.floor(8)); // 7 – largest element <= 8
System.out.println(ts.ceiling(8)); // 9 – smallest element >= 8
System.out.println(ts.lower(8)); // 7 – largest element strictly < 8
System.out.println(ts.higher(8)); // 9 – smallest element strictly > 8

// floor/ceiling return null if no such element
System.out.println(ts.floor(0)); // null
System.out.println(ts.ceiling(100)); // null

// pollFirst/pollLast – retrieve AND remove
System.out.println(ts.pollFirst()); // 1 (removed)
System.out.println(ts.pollLast()); // 15 (removed)
System.out.println(ts); // [3,5,7,9,11,13]

// Inclusive subSet and headSet
System.out.println(ts.subSet(3,true,9,true)); // [3,5,7,9] both inclusive
System.out.println(ts.headSet(9, true)); // [3,5,7,9] inclusive
System.out.println(ts.tailSet(7, false)); // [9,11,13] exclusive of 7

// Descending view – backed by same data, live view
NavigableSet<Integer> desc = ts.descendingSet();
System.out.println(desc); // [13,11,9,7,5,3]
```

4.3 SortedSet Views Are LIVE

```
TreeSet<Integer> ts = new TreeSet<>(Arrays.asList(1,3,5,7,9));
SortedSet<Integer> head = ts.headSet(6); // {1,3,5} – live view

// Modifying the view modifies the backing set
head.remove(3);
System.out.println(ts); // [1,5,7,9]

// Adding to backing set appears in view (if in range)
ts.add(4);
System.out.println(head); // [1,4,5] – automatically updated

// Adding out-of-range to view throws IllegalArgumentException
// head.add(7); // IllegalArgumentException: key out of range
```

Section 5 — EnumSet Deep Dive

5.1 Internal Representation — Bit Vectors

EnumSet is backed by a **bit vector** (one or more long values). Each enum constant's ordinal() maps directly to a bit position. This makes all operations O(1) with zero hash computation or tree traversal — just bitwise operations on primitive longs.

```
// Two concrete implementations (selected automatically by JDK):
// RegularEnumSet – uses a single long; for enums with <= 64 constants
// JumboEnumSet – uses long[] array; for enums with > 64 constants
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN } // ordinals 0-6
// Factory methods – always use these, never 'new EnumSet()'
EnumSet<Day> workdays = EnumSet.of(Day.MON, Day.TUE, Day.WED, Day.THU, Day.FRI);
EnumSet<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
EnumSet<Day> all = EnumSet.allOf(Day.class);
EnumSet<Day> none = EnumSet.noneOf(Day.class);
EnumSet<Day> notWeekend = EnumSet.complementOf(weekend); // {MON,TUE,WED,THU,FRI}
EnumSet<Day> midweek = EnumSet.range(Day.TUE, Day.THU); // {TUE,WED,THU}
// copyOf – from another EnumSet or non-empty EnumSet-typed collection
EnumSet<Day> copy = EnumSet.copyOf(workdays);
// Bit vector internals – workdays = 0b0011111 (bits 0-4 set for MON-FRI)
// contains(Day.MON) => (bits & (1L << Day.MON.ordinal())) != 0
// => (0b0011111 & (1L << 0)) != 0 => true
// Memory: 7 enum constants fit in ONE long (8 bytes)
// vs HashSet: ~4KB overhead for same 7 elements
```

5.2 EnumSet Performance and Usage

```
// Iteration in EnumSet follows enum declaration order (ordinal order)
EnumSet<Day> days = EnumSet.of(Day.FRI, Day.MON, Day.WED);
for (Day d : days) { System.out.print(d + " "); }
// Output: MON WED FRI (ordinal order, NOT insertion order)
// No nulls allowed
// days.add(null); // NullPointerException
// Use EnumSet for permission/flag modeling
enum Permission { READ, WRITE, EXECUTE, DELETE }
EnumSet<Permission> userPerms = EnumSet.of(Permission.READ, Permission.WRITE);
EnumSet<Permission> adminPerms = EnumSet.allOf(Permission.class);
boolean canWrite = userPerms.contains(Permission.WRITE); // 0(1) bit test
userPerms.add(Permission.EXECUTE);
userPerms.remove(Permission.DELETE); // no-op (not present)
// EnumSet.copyOf() requires argument to be a non-empty EnumSet or enum-typed Collection
// Cannot copyOf() an empty non-EnumSet collection because element type unknown at runtime
// EnumSet.copyOf(Collections.emptyList()); // throws IllegalArgumentException
// Solution: EnumSet.noneOf(Day.class) for empty
```

5.3 EnumSet Best Practice

ALWAYS prefer EnumSet over HashSet when working with enum values. EnumSet is dramatically faster (bit operations vs hashing), uses far less memory, and iterates in enum declaration order. There is no reason to use HashSet.

Section 6 — Set Operations & Algebra

6.1 Classical Set Operations

```
Set<Integer> a = new HashSet<>(Arrays.asList(1,2,3,4,5));
Set<Integer> b = new HashSet<>(Arrays.asList(3,4,5,6,7));
// UNION – all elements from both sets
Set<Integer> union = new HashSet<>(a);
union.addAll(b);
System.out.println(union); // [1,2,3,4,5,6,7]
// INTERSECTION – elements present in both sets
```

```

Set<Integer> intersection = new HashSet<>(a);
intersection.retainAll(b);
System.out.println(intersection); // [3,4,5]
// DIFFERENCE – elements in a but not in b
Set<Integer> difference = new HashSet<>(a);
difference.removeAll(b);
System.out.println(difference); // [1,2]
// SYMMETRIC DIFFERENCE – elements in a or b but not both
Set<Integer> symDiff = new HashSet<>(a);
symDiff.addAll(b); // union
Set<Integer> inter2 = new HashSet<>(a);
inter2.retainAll(b); // intersection
symDiff.removeAll(inter2); // union - intersection
System.out.println(symDiff); // [1,2,6,7]
// DISJOINT CHECK – do sets share no elements?
System.out.println(Collections.disjoint(a, b)); // false (share 3,4,5)
Set<Integer> c = new HashSet<>(Arrays.asList(8,9));
System.out.println(Collections.disjoint(a, c)); // true
// SUBSET CHECK – is a a subset of b?
Set<Integer> sub = new HashSet<>(Arrays.asList(3,4));
System.out.println(b.containsAll(sub)); // true – sub is a subset of b

```

6.2 Immutable Set Operations Using Streams

```

// Stream-based immutable set operations (Java 10+)
Set<Integer> a = Set.of(1,2,3,4,5);
Set<Integer> b = Set.of(3,4,5,6,7);
// Immutable union
Set<Integer> union = Stream.concat(a.stream(), b.stream())
    .collect(Collectors.toUnmodifiableSet());
// Immutable intersection
Set<Integer> intersection = a.stream()
    .filter(b::contains)
    .collect(Collectors.toUnmodifiableSet());
// Immutable difference
Set<Integer> difference = a.stream()
    .filter(e -> !b.contains(e))
    .collect(Collectors.toUnmodifiableSet());
// Set.copyOf() – defensive immutable copy (Java 10+)
Set<String> mutable = new HashSet<>(Arrays.asList("x","y","z"));
Set<String> defensiveCopy = Set.copyOf(mutable);
// mutable.add("w"); // only affects mutable, not defensiveCopy

```

Section 7 — Thread Safety & Concurrent Sets

7.1 HashSet Is NOT Thread-Safe

HashSet, LinkedHashSet, and TreeSet are all NOT thread-safe. Concurrent modification from multiple threads without synchronization can corrupt internal state, cause infinite loops (pre-Java 8 HashMap resize), or produce silently wrong results.

```

// Option 1: Collections.synchronizedSet – wraps with synchronized blocks
Set<String> syncSet = Collections.synchronizedSet(new HashSet<>());
syncSet.add("a"); // thread-safe for individual ops
// WARNING: Iteration still requires external synchronization!
synchronized (syncSet) {
    Iterator<String> it = syncSet.iterator();

```



```

while (it.hasNext()) {
    System.out.println(it.next()); // must hold lock during iteration
}
}

// Option 2: ConcurrentSkipListSet – concurrent NavigableSet
// No locking (CAS-based), O(log n), sorted, thread-safe for all operations including iteration
import java.util.concurrent.ConcurrentSkipListSet;
ConcurrentSkipListSet<Integer> skipSet = new ConcurrentSkipListSet<>();
skipSet.add(3); skipSet.add(1); skipSet.add(5);
System.out.println(skipSet); // [1,3,5] – sorted, thread-safe

// Option 3: CopyOnWriteArraySet – for tiny, read-heavy sets
import java.util.concurrent.CopyOnWriteArraySet;
CopyOnWriteArraySet<String> cowSet = new CopyOnWriteArraySet<>();
cowSet.add("x"); cowSet.add("y");
// Write creates a new backing array; reads are unsynchronized
// Iterator is a snapshot – never throws ConcurrentModificationException
for (String s : cowSet) {
    cowSet.add("z"); // OK – won't affect current iteration
}

// Option 4: ConcurrentHashMap.newKeySet() – best general-purpose concurrent set
import java.util.concurrent.ConcurrentHashMap;
Set<String> concSet = ConcurrentHashMap.newKeySet();
concSet.add("a"); concSet.add("b"); // fully thread-safe, O(1) avg

```

7.2 Thread Safety Comparison Table

Implementation	Thread-Safe	Ordering	Null	add/contains
HashSet	No	None	One null	O(1) avg
LinkedHashSet	No	Insertion	One null	O(1) avg
TreeSet	No	Sorted	No null	O(log n)
EnumSet	No	Enum ordinal	No null	O(1) bit op
synchronizedSet(HashSet)	Yes (coarse)	None	One null	O(1) avg
ConcurrentSkipListSet	Yes (CAS)	Sorted	No null	O(log n)
CopyOnWriteArraySet	Yes (copy-on-write)	Insertion	Allowed	O(n)
ConcurrentHashMap.newKeySet() ()	Yes (segment)	None	No null	O(1) avg

Section 8 — Choosing the Right Set

8.1 Decision Table

Use Case	Recommended Set
Fast uniqueness check, no ordering needed	HashSet
Unique elements preserving insertion order	LinkedHashSet
Sorted unique elements, range queries	TreeSet
Working with enum constants efficiently	EnumSet
Concurrent reads/writes, sorted order	ConcurrentSkipListSet
Concurrent reads/writes, unordered (general)	ConcurrentHashMap.newKeySet()
Mostly read, rarely written, small set	CopyOnWriteArraySet
Immutable set (Java 9+)	Set.of() or Set.copyOf()
Synchronize existing HashSet (legacy)	Collections.synchronizedSet()

8.2 Common Anti-Patterns

- **Using ArrayList for uniqueness:** `ArrayList.contains()` is $O(n)$; adding to an `ArrayList` and checking uniqueness is $O(n^2)$. Always use `HashSet` when uniqueness matters.
- **Missing equals/hashCode override:** `HashSet` treats objects as duplicates only if `equals()` AND `hashCode()` agree. Forgetting either causes silent duplicate storage.
- **Mutable objects as Set elements:** Mutating an object's fields after adding it to a `HashSet` changes its `hashCode`, making it invisible to the Set's lookup.
- **Using HashSet when TreeSet needed for ordering:** Converting `HashSet` to sorted list with `Collections.sort()` on every read is $O(n \log n)$ repeated — maintain a `TreeSet` instead.
- **Using HashSet instead of EnumSet:** `EnumSet` is 100x more memory efficient and faster for enum values.
- **Inconsistent Comparator>equals in TreeSet:** If `Comparator` says two objects are equal (returns 0) but `equals()` returns false, `TreeSet` silently treats them as duplicates — one will be lost.

Q&A; — Standard Questions (Q1–Q35)

Q1. How does HashSet ensure uniqueness of elements?

HashSet uses a HashMap internally. When `add(e)` is called, it invokes `map.put(e, PRESENT)`. `HashMap.put()` first computes `e.hashCode()` to find the bucket, then checks existing entries in that bucket using `equals()`. If any existing key equals `e`, the entry is overwritten (no new entry added) and `put()` returns the old value. `HashSet.add()` returns `false` in this case. So uniqueness is enforced through the combined `hashCode` (bucket selection) + `equals` (identity check) contract.

Q2. What happens if two objects have the same hashCode but different equals() in a HashSet?

They are stored as separate entries in the same bucket (hash collision). The HashSet correctly stores both elements because `equals()` returns `false` — they are not duplicates. Performance degrades to $O(k)$ where k is the collision chain length, but correctness is maintained. In Java 8+, if a bucket's chain exceeds a threshold (8 entries), it is treeified to $O(\log n)$.

Q3. Why can HashSet have at most one null, but TreeSet cannot have any null?

HashSet maps null to a special bucket (index 0 in HashMap — null keys are handled specially). Since null is treated like any other key, only one null is allowed (duplicates are rejected). TreeSet uses a Comparator or Comparable to order elements. Calling `compareTo(null)` throws `NullPointerException`, and `compareTo()` is called on every add. Thus TreeSet cannot store null. Even if a null-safe Comparator is provided, the natural ordering `compareTo` contract forbids null.

Q4. When does TreeSet throw ClassCastException?

TreeSet throws `ClassCastException` when you add an element that cannot be compared to existing elements. This happens when: (1) the element's class does not implement `Comparable`, and no Comparator was provided at TreeSet construction; or (2) you add an incompatible type (e.g., adding an Integer to a TreeSet). The exception is thrown at `add()` time when `compareTo()` is invoked. Note: if you add only ONE element to a TreeSet with non-Comparable objects and no Comparator, no exception is thrown (no comparison needed) — it only fails on the second add.

Q5. Explain the difference between headSet/tailSet inclusive vs exclusive behavior.

In `SortedSet`: `headSet(toElement)` is exclusive of `toElement`; `tailSet(fromElement)` is inclusive. So `headSet(6)` gives elements strictly less than 6, `tailSet(6)` gives elements ≥ 6 . In `NavigableSet` (TreeSet): `headSet(toElement, inclusive)` and `tailSet(fromElement, inclusive)` accept a boolean to control inclusivity. `headSet(6, true)` includes 6 if present; `headSet(6, false)` excludes it. Similarly, `subSet(from, fromIncl, to, toIncl)` gives full control. Default `subSet(from, to)` is `fromInclusive=true, toInclusive=false`.

Q6. How does LinkedHashSet maintain insertion order?

`LinkedHashSet` uses a `LinkedHashMap` as its backing store. `LinkedHashMap.Entry` extends `HashMap.Node` and adds two extra pointers: 'before' and 'after', forming a doubly-linked list through all entries in insertion order. When an element is added, a new Entry is linked at the tail of this list. When an element is removed, the linked list is updated by relinking its neighbors. The iterator traverses the linked list, not the bucket array, giving insertion-order iteration. Re-inserting an existing element does NOT move it in the linked list.

Q7. Describe EnumSet's internal representation and why it is faster than HashSet.

`EnumSet` uses a bit vector backed by one or more long primitives. Each enum constant has an `ordinal()` value (0, 1, 2...). `EnumSet` stores element presence as bits: bit position i is 1 if the enum with ordinal i is in the set. `add(e)` does: `elements |= (1L << e.ordinal())`. `contains(e)` does: `(elements & (1L << e.ordinal())) != 0`. These are single CPU instruction bitwise operations — no hashing, no object allocation, no bucket traversal. HashSet requires computing `hashCode()`, finding a bucket, traversing the chain, and calling `equals()` — all with heap object accesses. EnumSet is 10-100x faster for `add/contains`.

Q8. What is NavigableSet and which methods extend SortedSet?

NavigableSet extends SortedSet with navigation methods for closest-match queries. SortedSet provides: `first()`, `last()`, `headSet(to)`, `tailSet(from)`, `subSet(from,to)`, `comparator()`. NavigableSet adds: `lower(e)` — largest element strictly less than `e`; `floor(e)` — largest element less than or equal to `e`; `ceiling(e)` — smallest element greater than or equal to `e`; `higher(e)` — smallest element strictly greater than `e`; `pollFirst()` — retrieve and remove first; `pollLast()` — retrieve and remove last; `descendingSet()` — reverse-order view; `descendingIterator()`; inclusive `headSet/tailSet/subSet` overloads. `TreeSet` and `ConcurrentSkipListSet` implement `NavigableSet`.

Q9. How do you get the 'closest' element in a TreeSet using floor() and ceiling()?

`floor(e)` returns the greatest element in the set that is less than or equal to `e`, or null if no such element exists. `ceiling(e)` returns the least element greater than or equal to `e`, or null. Example: `TreeSet ts = new TreeSet<>(Arrays.asList(1,3,5,7,9));` `ts.floor(6)` returns 5 (largest ≤ 6); `ts.ceiling(6)` returns 7 (smallest ≥ 6); `ts.floor(1)` returns 1 (exact match); `ts.floor(0)` returns null (nothing ≤ 0). These are $O(\log n)$ operations and extremely useful for range-query problems like finding the nearest number, implementing order books, or range-based lookups.

Q10. What is the difference between pollFirst() and first() in TreeSet?

`first()` returns the smallest element without removing it — it is a non-destructive peek. `pollFirst()` retrieves AND removes the smallest element — it is destructive. If the set is empty, `first()` throws `NoSuchElementException`, while `pollFirst()` returns null. Similarly, `last()` vs `pollLast()` for the largest element. `pollFirst()/pollLast()` are useful for implementing priority queues, sliding windows, or processing elements in sorted order (like a sorted queue).

Q11. How do you create an immutable Set? Name 3 ways.

(1) `Set.of(elements)` — Java 9+: `Set s = Set.of("a","b","c");` throws `UnsupportedOperationException` on mutation, `NullPointerException` on null elements. (2) `Collections.unmodifiableSet(set)` — wraps an existing `Set`: `Set s = Collections.unmodifiableSet(mutableSet);` mutations throw `UnsupportedOperationException`, but the underlying `mutableSet` can still be modified externally. (3) `Set.copyOf(collection)` — Java 10+: takes a snapshot, immutable, rejects nulls. A fourth option: `Collections.singleton(e)` for a single-element immutable `Set`.

Q12. Compare Set.of() vs Collections.unmodifiableSet() vs Collections.singleton().

`Set.of()` (Java 9+): creates a truly immutable set from scratch; rejects nulls; implementation may shuffle element storage for security; supports 0-10 elements with specialized implementations, and varargs for larger sets. `Collections.unmodifiableSet()`: wraps an existing mutable `Set` in an unmodifiable view; does NOT copy — if the original set is modified, the unmodifiable view reflects the change; allows one null (if backing set allows it). `Collections.singleton(e)`: single-element immutable `Set`; allows null as the element; all mutation methods throw `UnsupportedOperationException`.

Q13. How do you sort a Set (which is inherently unordered for HashSet)?

Three approaches: (1) Convert to a sorted collection: `List sorted = new ArrayList<>(hashSet); Collections.sort(sorted);` — $O(n \log n)$. (2) Use a `TreeSet` instead: `new TreeSet<>(hashSet)` gives a sorted view — $O(n \log n)$ one-time cost, then $O(\log n)$ per operation. (3) Use streams: `hashSet.stream().sorted().collect(Collectors.toList())` — returns a sorted `List`. Note: if you frequently need sorted traversal, switch to `TreeSet` from the start rather than sorting repeatedly. `LinkedHashSet` preserves insertion order but not sorted order.

Q14. How do you convert a Set to a sorted List?

Three clean ways: (1) `new ArrayList<>(set)` then sort: `List list = new ArrayList<>(set); Collections.sort(list);` (2) `TreeSet` to `List`: `new ArrayList<>(new TreeSet<>(set))` — `TreeSet` handles sorting, then wrap in `ArrayList`. (3) Stream: `List list = set.stream().sorted().collect(Collectors.toList());` For custom order: `set.stream().sorted(Comparator.reverseOrder()).collect(Collectors.toList());` All three are $O(n \log n)$. If you need case-insensitive: `.sorted(String.CASE_INSENSITIVE_ORDER)`.

Q15. Compare containsAll() performance: HashSet vs TreeSet vs ArrayList.

Checking if set/collection A containsAll(B) iterates B and calls A.contains() for each element. HashSet.contains() is $O(1)$ avg, so containsAll() on HashSet checking B elements is $O(|B|)$. TreeSet.contains() is $O(\log n)$, so containsAll() is $O(|B| * \log |A|)$. ArrayList.contains() is $O(n)$, so containsAll() is $O(|B| * |A|)$ — very slow for large collections. Practical implication: to check if all elements of a List exist in a collection, put the collection in a HashSet first for $O(n)$ total vs $O(n^2)$ for nested ArrayList checks.

Q16. Describe CopyOnWriteArraySet iterator behavior.

CopyOnWriteArraySet is backed by a CopyOnWriteArrayList. Its iterator takes a snapshot of the backing array at the time the iterator is created. The iterator operates on this snapshot, so: (1) The iterator NEVER throws ConcurrentModificationException — it's a snapshot, no modCount tracking. (2) Modifications to the set after iterator creation are NOT visible through the current iterator. (3) The iterator does NOT support the remove() operation (throws UnsupportedOperationException). This makes it safe for concurrent iteration but means the iterator may see stale data.

Q17. How does Set equality work between a HashSet and TreeSet containing the same elements?

Set equality is defined in AbstractSet.equals(): two sets are equal if they have the same size and one containsAll() the other. The specific implementation (HashSet vs TreeSet) does NOT matter. Example: `HashSet hs = new HashSet<>(Arrays.asList(1,2,3)); TreeSet ts = new TreeSet<>(Arrays.asList(1,2,3)); System.out.println(hs.equals(ts)); // true` The equals() check uses each set's contains() method, which relies on hashCode/equals for HashSet and compareTo for TreeSet. If elements correctly implement these, cross-implementation equality works correctly.

Q18. What is the difference between subSet(a, b) and subSet(a, true, b, true)?

subSet(a, b) from SortedSet is equivalent to subSet(a, true, b, false) — fromElement a is inclusive, toElement b is exclusive. This means elements in range [a, b). subSet(a, true, b, true) from NavigableSet includes both endpoints — range [a, b]. subSet(a, false, b, false) excludes both — range (a, b). Example with TreeSet containing {1,3,5,7,9}: subSet(3, 7) returns {3,5} (7 excluded); subSet(3, true, 7, true) returns {3,5,7} (7 included).

Q19. Why does EnumSet.copyOf(collection) require the collection to be a non-empty EnumSet?

EnumSet needs to know the enum type at runtime to create the correct EnumSet instance (RegularEnumSet or JumboEnumSet) and to validate elements. When the argument is an EnumSet, the element type is embedded in the EnumSet instance. For a general Collection, the element type is erased at runtime (type erasure), so EnumSet cannot determine which enum class to use. If you have a non-EnumSet collection of enum values, iterate it and add manually, or use: `EnumSet result = EnumSet.noneOf(Day.class); result.addAll(collection);`

Q20. Can you implement a Set that maintains both insertion order AND sorted order simultaneously? If not, what are the workarounds?

No standard Set implementation does both simultaneously — LinkedHashSet preserves insertion order, TreeSet preserves sorted order, but neither does both. Workarounds: (1) Maintain two separate structures: a LinkedHashSet for insertion-order iteration and a TreeSet for sorted iteration — adds $O(n)$ memory overhead; wrap in a custom class to keep them in sync. (2) Use a sorted insertion structure: always insert into a position-aware sorted list (TreeMap with insertion-index values) and build a sorted-by-insertion or sorted-by-value view as needed. (3) Accept a compromise: if elements are Comparable, collect into a List and sort when needed. (4) Third-party: Guava's ImmutableSortedSet preserves both for a snapshot.

Q21. What is the time complexity of add() in TreeSet when adding n elements one by one?

Each add() in a TreeSet is $O(\log k)$ where k is the current number of elements. Adding n elements one by one is $O(n \log n)$ total. Internally, TreeSet's TreeMap rebalances the Red-Black tree after each insert, which is $O(\log n)$ worst case. Comparison: HashSet add() is $O(1)$ amortized per element, so $O(n)$ total for n elements. If you have a sorted array/collection and want to build a TreeSet, the constructor TreeSet(Collection c) uses addAll which is still $O(n \log n)$. There is no $O(n)$ bulk construction shortcut for TreeSet (unlike sorted arrays which can be converted in $O(n)$).

Q22. What happens when you iterate over a HashSet and call remove() during iteration?

Calling `set.remove(element)` directly while iterating with a for-each loop causes `ConcurrentModificationException` on the next call to `iterator.next()` because `HashSet`'s iterator is fail-fast: it checks `modCount` before each `next()` call. The correct approach is to use `Iterator` explicitly: `Iterator it = set.iterator(); while (it.hasNext()) { String s = it.next(); if (condition) { it.remove(); } } Iterator.remove()` removes the current element without incrementing `modCount` for the iterator. Alternatively, use `removeIf()` (Java 8+): `set.removeIf(s -> condition)`.

Q23. How does HashSet resize, and what is the load factor's role?

`HashSet` (via `HashMap`) resizes when the number of elements exceeds `capacity * loadFactor`. Default capacity is 16, default load factor is 0.75, so resizing occurs at 12 elements. Resizing doubles the capacity (to 32, 64, etc.) and rehashes all existing entries into new buckets. This is $O(n)$ and causes a temporary performance spike. A lower load factor (e.g., 0.5) reduces collisions but uses more memory and resizes more often. A higher factor (e.g., 0.9) saves memory but increases collisions. If you know the expected size, pre-size: `new HashSet<>(expectedSize / 0.75 + 1)` to avoid all resizes.

Q24. What is the difference between Set.of() in Java 9+ and new HashSet()?

`Set.of()` creates an immutable, fixed-size set with no nulls allowed. The implementation is not a `HashSet` — it uses specialized compact representations (`SetN` for variable size, `Set0`, `Set1`, `Set2` for small sizes) that are more memory-efficient. Mutation throws `UnsupportedOperationException`. Duplicate elements at creation throw `IllegalArgumentException`. `new HashSet<>()` is mutable, allows null, and allocates a `HashMap` internally. `Set.of()` is ideal for constants, configuration, and defensive copies. `new HashSet<>()` is for mutable collections used in business logic.

Q25. How would you find the symmetric difference of two sets in Java?

Symmetric difference = (A union B) minus (A intersection B) — elements in A or B but not both. Implementation: `Set symDiff = new HashSet<>(a); symDiff.addAll(b); Set intersection = new HashSet<>(a); intersection.retainAll(b); symDiff.removeAll(intersection);` Or using streams: `Set symDiff = Stream.concat(a.stream().filter(e -> !b.contains(e)), b.stream().filter(e -> !a.contains(e))).collect(Collectors.toSet());` The stream version is more readable and works with immutable inputs.

Q26. What is ConcurrentSkipListSet and when should you use it over synchronizedSet?

`ConcurrentSkipListSet` implements `NavigableSet` and is backed by a `ConcurrentSkipListMap`. It is a concurrent, thread-safe, sorted set using a probabilistic skip list data structure with CAS (Compare-And-Swap) operations — no global lock. Use it when: (1) you need a sorted, thread-safe set with good concurrent throughput; (2) multiple threads read and write simultaneously (unlike `synchronizedSet` which has one lock); (3) you need `NavigableSet` operations (floor, ceiling, headSet) in a concurrent context. `synchronizedSet` wraps with a single mutex — only one thread at a time for any operation. `ConcurrentSkipListSet` allows concurrent reads and writes with $O(\log n)$ operations.

Q27. What is the contract for equals() in sets — how is Set equality defined?

`AbstractSet.equals()` defines Set equality: two sets are equal if they are both `Set` instances, have the same size, and one containsAll the other (which means they contain exactly the same elements). The type of set (`HashSet` vs `TreeSet`) does not matter — only the elements matter. Importantly, `AbstractSet.hashCode()` is defined as the sum of `hashCode`s of all elements, ensuring that equal sets have equal `hashCode`s (required by the `hashCode` contract). This means a `HashSet` and a `TreeSet` with the same elements are `equals() == true` and have the same `hashCode`.

Q28. How do you check if two sets are disjoint (share no common elements) efficiently?

Use `Collections.disjoint(set1, set2)`. It is optimized: if `set1` is a `Set`, it iterates `set2` and calls `set1.contains()` for each element, giving $O(|set2|)$ for `HashSet` (`contains` is $O(1)$). If `set2` is smaller, the implementation may swap them. For manual implementation: `boolean disjoint = set1.stream().noneMatch(set2::contains);` — equivalent but less optimized. Do NOT convert to a `List` and use `List.contains()` — that is $O(n)$ per check, $O(n^2)$ total.

Q29. How does TreeSet handle custom objects? What must the custom class provide?

For TreeSet to work with custom objects, the class must either: (1) implement Comparable and override compareTo() to define natural ordering; or (2) a Comparator must be provided to the TreeSet constructor. If neither is done, adding the second element throws ClassCastException. CRITICAL: If a Comparator is used, it should be consistent with equals() — comparator.compare(a, b) == 0 should imply a.equals(b) and vice versa. If they are inconsistent, TreeSet may silently drop elements it considers equal by compareTo but are not equal by equals(), or contains() may return false for elements that are present.

Q30. What is the memory cost of a HashSet compared to an array for 1000 integers?

Array int[1000]: 4 bytes * 1000 = ~4KB. HashSet: each Integer object is 16 bytes (object header + int field). Each HashMap.Node (bucket entry) is ~32 bytes (hash int, key ref, value ref, next ref). With default load factor 0.75, capacity is ~1334 buckets -> 1334 * 4 bytes (reference array) = ~5KB. Total: ~16KB for Integer objects + ~32KB for nodes + ~5KB for array = ~53KB. This is ~13x more than a primitive array. For int values, EnumSet is impossible (only for enums). BitSet could store 1000 integers in ~125 bytes if values are bounded. This overhead is why primitives are preferred when ordering/uniqueness-checking on massive int arrays.

Q31. What is the difference between remove(Object o) and removeIf(Predicate) in Set?

remove(Object o) removes a specific element by value — requires an exact element to remove. Calls equals() to find the element. Returns true if the element was present and removed. removeIf(Predicate filter) is a Java 8+ default method that removes all elements matching the predicate. It uses the iterator internally (with iterator.remove()) to safely remove during iteration, avoiding ConcurrentModificationException. Example: set.removeIf(s -> s.startsWith("A")); removes all strings starting with A. removeIf is more powerful — it handles bulk conditional removal in one readable call.

Q32. Can a Set contain itself as an element? What would happen?

Technically, you can do: Set s = new HashSet<>(); s.add(s); This causes s.hashCode() to call AbstractSet.hashCode() which sums hashCodes of all elements — but one element IS s itself, causing infinite recursion and StackOverflowError. s.toString() would also overflow. s.equals(s) would call containsAll(s) which iterates s while s contains s — potentially infinite. The Java documentation for AbstractSet specifically notes this as undefined behavior: 'it is not permissible for a set to contain itself as an element.'

Q33. How would you implement a method to remove duplicate strings from a List while preserving order?

Use LinkedHashSet: public List removeDuplicates(List list) { return new ArrayList<>(new LinkedHashSet<>(list)); } This is O(n) average: LinkedHashSet constructor iterates the list adding each element (O(1) per element), then new ArrayList wraps the result. Insertion order (first occurrence) is preserved. Stream alternative: list.stream().distinct().collect(Collectors.toList()) — also preserves first occurrence. Using HashSet loses ordering. Using TreeSet sorts but does not preserve insertion order.

Q34. What is the fail-fast behavior of HashSet's iterator?

HashSet inherits its iterator from HashMap.keySet().iterator(). This iterator is fail-fast: it maintains a modCount snapshot at creation. Before each next() call, it checks if the set's current modCount equals its snapshot. If any structural modification (add, remove, clear — any operation that changes size or structure) occurs on the set after the iterator is created (and not via iterator.remove()), modCount changes and the next next() call throws ConcurrentModificationException. This is a best-effort mechanism — it is not guaranteed to fire in all concurrent cases, so it should not be relied upon for correctness in multithreaded code.

Q35. How does EnumSet.range() work, and what is its use case?

EnumSet.range(from, to) creates an EnumSet containing all enum constants from 'from' to 'to' inclusive, based on their ordinal() order (the order in which they are declared in the enum). Example: enum Day { MON, TUE, WED, THU, FRI, SAT, SUN } EnumSet.range(Day.MON, Day.FRI) returns {MON, TUE, WED, THU, FRI} — ordinals 0-4. It throws IllegalArgumentException if from.ordinal() > to.ordinal(). Internally, it sets bits from from.ordinal() to to.ordinal() using a bitmask. Use cases: day ranges, month ranges, priority ranges, permission levels — any contiguous enum range that needs efficient set operations.

Q&A; — Advanced Questions (Q36–Q52)

[ADV] Q36. Explain HashSet's internal PRESENT dummy object. Why use HashMap rather than a custom implementation?

HashSet stores elements as HashMap keys with a shared static final `PRESENT = new Object()` as the value. This dummy object serves no purpose other than satisfying HashMap's value requirement. Why reuse HashMap? (1) Code reuse and maintenance: HashMap is a highly optimized, well-tested structure. Reusing it means HashSet automatically inherits all HashMap improvements (Java 8 treeification, Java 9 compact hashing, SIMD-friendly layouts in future JVMs). (2) Reduced bug surface: a custom open-addressing hash set would need to re-implement probe sequences, tombstone handling, and resize logic — all sources of subtle bugs. (3) Tradeoff: the PRESENT object wastes one reference per entry (~8 bytes) and one extra field in each `HashMap.Node`. A custom implementation could store elements more compactly, but the JDK prioritizes correctness and shared maintenance over this minor memory overhead. Guava's `HashMultiset` and Eclipse Collections offer more specialized options.

[ADV] Q37. Explain why TreeSet guarantees $O(\log n)$ worst-case while HashMap had $O(n)$ worst-case before Java 8.

TreeSet uses a Red-Black tree: a self-balancing BST that maintains the invariant that no path from root to leaf is more than 2x longer than any other. Rebalancing after insert/delete is $O(\log n)$ rotations. This guarantees height $\leq 2 \cdot \log_2(n+1)$, ensuring $O(\log n)$ worst-case for all operations. Pre-Java 8 HashMap used linked-list chaining. A malicious or poor hash function could place all n elements in the same bucket, making the chain $O(n)$ for contains/remove. Java 8 introduced treeification: when a bucket's chain exceeds `TREEIFY_THRESHOLD` (8) AND table size $\geq \text{MIN_TREEIFY_CAPACITY}$ (64), the chain becomes a Red-Black tree, restoring $O(\log n)$ per-bucket worst case. Below 64 table size, the table is resized instead of treeifying.

[ADV] Q38. Explain EnumSet's bit vector internals: ordinal mapping, complementOf, RegularEnumSet vs JumboEnumSet.

Each enum constant e has a unique ordinal: `e.ordinal() = 0, 1, 2, ...` in declaration order. `RegularEnumSet` (for enums with ≤ 64 constants) stores one long 'elements'. `add(e)`: `elements |= (1L << e.ordinal())`. `contains(e)`: `(elements & (1L << e.ordinal())) != 0`. `remove(e)`: `elements &= ~(1L << e.ordinal())`. `size()`: `Long.bitCount(elements)`. `complementOf(s)`: new `EnumSet` with `(~s.elements) & mask`, where mask has bits set for valid ordinals. `JumboEnumSet` (for enums > 64 constants) uses `long[]` elements array. Element e maps to `elements[e.ordinal() / 64] bit (e.ordinal() % 64)`. Same bitwise logic but across multiple longs. Memory: `RegularEnumSet` for 64-enum type = 1 long = 8 bytes. `HashSet` for same 64 elements = ~64 `HashMap.Node` objects = ~2KB minimum.

[ADV] Q39. How does ConcurrentSkipListSet differ from TreeSet in concurrent scenarios?

TreeSet (backed by `TreeMap`) uses no synchronization — concurrent access causes data corruption or `ConcurrentModificationException`. External locking serializes all access. `ConcurrentSkipListSet` (backed by `ConcurrentSkipListMap`) uses a probabilistic skip list data structure with CAS (Compare-And-Swap) operations at the node level. A skip list is a layered linked list: the bottom layer contains all elements in sorted order; upper layers contain random subsets forming 'express lanes' for $O(\log n)$ traversal. CAS avoids global locking — multiple threads can concurrently add/remove elements to different parts of the skip list. Expected $O(\log n)$ per operation. Weakly consistent iterator: iterates elements as of some point, never throwing `ConcurrentModificationException`, potentially reflecting some but not all writes after iterator creation. Unlike TreeSet, `ConcurrentSkipListSet` supports concurrent bulk operations (`addAll`) and does not lock.

[ADV] Q40. Describe LinkedHashMap.Entry's memory layout and the overhead LinkedHashMap adds over HashSet.

`HashMap.Node` has four fields: `int hash`, `K key`, `V value`, `Node next`. On a 64-bit JVM with compressed oops: object header=12 bytes, hash=4, key=4, value=4, next=4 = 28 bytes, padded to 32 bytes. `LinkedHashMap.Entry` extends `Node` and adds: `Entry before`, `Entry after` — two more reference fields = 8 additional bytes, total 40 bytes per entry (vs 32 for `HashMap.Node`). So `LinkedHashSet` uses ~25% more memory per element than `HashSet`. For 1 million elements: `HashSet` ~32MB per node storage, `LinkedHashSet` ~40MB. The linked list pointers are maintained on every add and remove, adding constant-factor overhead but not changing asymptotic complexity.

[ADV] Q41. Why does Set.of() in Java 9+ shuffle elements using a random seed? What attack does this prevent?

Java 9+ Set.of() (and Map.of()) uses a randomized seed (chosen at JVM startup) to determine internal element placement. This defends against hash flooding DoS attacks. Hash flooding: an attacker crafts input strings that all map to the same hash bucket in a deterministic hash map, degrading $O(1)$ to $O(n)$ per lookup and causing CPU exhaustion. Example: if hashCode() is deterministic, an attacker can precompute thousands of strings with the same hashCode and send them as Set.of() keys in a web request. The random seed (SALT) is mixed into the hash computation: $\text{hash} = \text{element.hashCode()} \oplus \text{SALT}$ — the attacker cannot predict which strings collide because SALT is unknown and changes each JVM run. Note: regular HashMap/HashSet are NOT randomized — this protection applies only to the immutable Set.of() / Map.of() implementations.

[ADV] Q42. What Spliterator characteristics does HashSet vs TreeSet report, and how does this impact parallel streams?

HashSet.spliterator() reports: DISTINCT (no duplicates) and SIZED (known size). It does NOT report ORDERED (no guaranteed order) or SORTED. Parallel streams over HashSet can freely reorder and reassign elements to threads — good for pure aggregations (count, sum) but stream operations that depend on order may produce unpredictable results (acceptable for sets since order is not semantically meaningful). TreeSet.spliterator() reports: DISTINCT, SIZED, SORTED (has a Comparator), and ORDERED (iteration order defined). The SORTED characteristic enables stream optimizations: sorted() is a no-op, min()/max() can use first/last without full scan, and forEachRemaining() can leverage sorted traversal. ORDERED limits parallel stream efficiency since some operations must respect element order. EnumSet.spliterator() reports DISTINCT, IMMUTABLE (for noneOf etc.), NONNULL, SIZED, ORDERED, SORTED.

[ADV] Q43. Compare memory usage of HashSet vs EnumSet for 64 enum values.

Enum type with 64 constants. EnumSet (RegularEnumSet): stores a single long field (64 bits = 8 bytes). Plus the EnumSet object header (~24 bytes). Total: ~32 bytes regardless of how many of the 64 constants are in the set. HashSet: for all 64 enum constants present — $64 * (16 \text{ bytes enum obj} + 32 \text{ bytes HashMap.Node}) = 64 * 48 = 3072 \text{ bytes}$. Plus HashMap internal array: next power of 2 $\geq 64/0.75 = 86$, so capacity=128, array=128*4=512 bytes. Plus HashSet/HashMap object headers ~48 bytes. Total: ~3.6KB. EnumSet is ~450x more memory-efficient. Performance: EnumSet bitwise ops vs HashSet hashing = typically 10-100x faster for contains/add. The contains() comparison is one AND instruction vs hashCode computation + bucket traversal + equals() call.

[ADV] Q44. How does modCount-based fail-fast work in AbstractSet, and why does removeAll use an element count heuristic?

AbstractCollection (parent of AbstractSet) maintains an int modCount incremented on every structural modification. Iterators capture the modCount at creation; next() throws ConcurrentModificationException if current modCount != captured value. AbstractSet.removeAll(c) has a subtle optimization: if $c.size() < this.size()$, iterate c and call this.remove(e) for each element in c. if $c.size() \geq this.size()$, iterate this via iterator and call c.contains(e) for each element. Why? If c is a tiny collection and 'this' is large, iterating c and calling this.remove() is $O(|c| * \text{remove cost})$. If c is huge and 'this' is small, iterating 'this' and calling c.contains() is $O(|this| * \text{contains cost})$. This heuristic chooses the cheaper direction. For HashSet this.remove() and c.contains() are both $O(1)$, so the dominant factor is which loop is shorter.

[ADV] Q45. Explain HashSet.removeAll() adaptive algorithm when the argument set is larger than 'this' set.

AbstractSet.removeAll(Collection c) overrides AbstractCollection.removeAll() with a smarter implementation. The code (simplified from JDK): if $(size() > c.size())$ { iterate c, call this.remove(e) } else { iterate this via iterator, if c.contains(e) call iterator.remove() } When c is larger than this: iterating c and calling this.remove() = $O(|c|)$ iterations but only $O(|this|)$ actual removals. If c is a HashSet, each c.contains() during the else-branch would be $O(1)$ but we'd iterate all |this| elements. Iterating c and calling this.remove() avoids iterating the large c. When this is larger: iterating this and using c.contains() avoids calling remove() on non-existent elements. This adaptive algorithm avoids $O(n*m)$ naive complexity and achieves $O(\min(n,m))$ iteration cycles.

[ADV] Q46. Explain the silent bug caused by Comparator inconsistent with equals in TreeSet.

TreeSet uses compareTo (or Comparator.compare) for ALL operations: add, remove, contains. It never calls equals(). If compareTo returns 0 for two objects that equals() says are different, TreeSet treats them as the same element and will not add the second one — silently dropping it. Example: `TreeSet ts = new TreeSet<>(String.CASE_INSENSITIVE_ORDER); ts.add("Apple"); ts.add("apple");` // 'apple' silently dropped! `ts.contains("apple")` returns true (found via comparator), but `ts.contains("Apple")` also true. The set has only 1 element but both case variants are 'contained'. The SortedSet contract says: 'A sorted set behaves as if equals() comparisons always return true when compareTo returns 0.' Violating this by using case-insensitive compare but case-sensitive equals produces a set that is inconsistent with the general Set contract. Always ensure comparator == equals.

[ADV] Q47. How does ConcurrentHashMap.newKeySet() work internally as a Set? Compare to Collections.synchronizedSet().

`ConcurrentHashMap.newKeySet()` returns a `KeySetView` backed by a `ConcurrentHashMap` where values are a shared `Boolean.TRUE` constant (similar to `HashSet`'s `PRESENT` object). `ConcurrentHashMap` uses segment-level locking (or CAS for read-heavy paths in Java 8+): the table is divided into segments; each segment has its own lock, allowing concurrent writes to different segments. Reads are lock-free using volatile reads. `Collections.synchronizedSet(set)` wraps the set in a `SynchronizedSet` which holds a single mutex (the set itself by default). Every operation — add, remove, contains, size, iterator — acquires this one mutex. Only one thread can perform any operation at a time — poor concurrent throughput. `ConcurrentHashMap.newKeySet()` allows concurrent reads and concurrent writes to different buckets simultaneously. Iterator is weakly consistent. Choice: use `synchronizedSet` for legacy single-thread patterns needing thread-safety. Use `newKeySet()` for any concurrent/high-throughput scenario.

[ADV] Q48. How does TreeSet handle the case when elements are added after creating a NavigableSet view?

`TreeSet` views (`headSet`, `tailSet`, `subSet`, `descendingSet`) are LIVE VIEWS backed by the same `TreeMap`. They share the same data structure, protected by range constraints. Adding an element to the backing `TreeSet` that falls within a view's range: the view immediately reflects the new element — its `size()` increases and the element appears in iteration. Adding an element to the view itself: the view validates the element against its range and throws `IllegalArgumentException` if the element is out of range. If in range, it is added to the backing set. Removing from the view removes from the backing set and vice versa for elements in the view's range. This live-view semantics is why `TreeSet` subsets are powerful for implementing range-based algorithms: you can maintain a sliding window by keeping a `subSet` view and adding/removing from the backing set.

[ADV] Q49. What happens to a HashSet when two elements have perfectly distributed hash codes but the same bucket after modulo?

This is normal hash collision — multiple keys can hash to the same bucket even with well-distributed hash codes. In Java 8+, each bucket starts as a linked list. After `TREEIFY_THRESHOLD` (8) entries in a bucket AND `table size >= MIN_TREEIFY_CAPACITY` (64), the bucket is converted to a `TreeNode` (Red-Black tree), providing $O(\log k)$ lookup within that bucket. The treeification threshold 8 was chosen because the Poisson distribution with default load factor 0.75 predicts less than 1-in-10-million chance of 8 entries in a bucket by chance. So in practice, treeification only triggers under hash flooding or severely broken `hashCode()`. When elements are removed and a treeified bucket shrinks below `UNTREEIFY_THRESHOLD` (6), it reverts to a linked list to save memory overhead of tree nodes.

[ADV] Q50. Explain how Set equality and hashCode work together in AbstractSet — and the performance implication.

`AbstractSet.equals(Object o)`: checks `o instanceof Set`, then `size() == o.size()`, then `this.containsAll(o)`. `containsAll()` iterates `o` and calls `this.contains()` for each element. For `HashSet`, `contains()` is $O(1)$, so total `equals()` is $O(|o|)$. For `TreeSet`, $O(|o| * \log(\text{this}))$. `AbstractSet.hashCode()`: defined as sum of `e.hashCode()` for all elements `e` in the set (nulls contribute 0). This ensures equal sets have equal `hashCode`s (consistent with the equals contract). Performance implication: iterating all elements to compute `hashCode` is $O(n)$. If you use a `Set` as a `Map` key or store sets in another `Set`, `hashCode` is called frequently. Solution: cache the `hashCode` if the set is immutable (`Set.of()` does this). Mutable sets cannot cache `hashCode` safely since elements can be added/removed. This is another reason to prefer immutable sets as map keys.

[ADV] Q51. How does Java's TreeSet implement the 'last less than or equal to X' query efficiently for interview problems?

TreeSet.floor(x) returns the greatest element less than or equal to x, in $O(\log n)$. This is implemented by navigating the Red-Black tree: start at root, if root.key $\leq x$, record it as candidate and go right (looking for larger valid candidate); if root.key $> x$, go left (must find smaller). Repeat until null. Classic interview applications: (1) Longest consecutive sequence: insert all numbers, for each n check if n-1 not in set ($O(n \log n)$ vs $O(n)$ with HashMap). (2) Minimum difference between array elements: insert into TreeSet, for each element x check floor(x-1) and ceiling(x+1) for nearest neighbors — $O(n \log n)$ total. (3) Sliding window maximum with eviction: maintain a TreeSet of (value, index) pairs, remove expired indices, query last() for max. TreeSet's NavigableSet operations map directly to efficient BST traversal — no sequential scan needed.

[ADV] Q52. What are the implications of using mutable objects as HashSet elements — both the bug and the detection strategy?

When a mutable object is added to a HashSet, its hashCode and equals() are evaluated at insertion time to determine its bucket. If the object's fields that contribute to hashCode are later mutated, the object's logical position in the HashSet becomes inconsistent: (1) contains(mutatedObj) returns false — different bucket is checked. (2) The object is 'lost' — it exists in the set's bucket array at its original bucket but is unreachable via normal lookup. (3) size() still counts it, but it cannot be removed via remove(mutatedObj). (4) Iteration will still yield it (bucket scan finds all entries regardless of hash consistency). Detection strategy: (a) Use assertion: before mutating, remove from set, mutate, re-add. (b) Use immutable value objects (Java records, final fields). (c) Run integrity checks: set.stream().filter(e -> !set.contains(e)).count() > 0 indicates corruption. Best practice: objects used as Set elements should be effectively immutable or use identity-based hashCode (default Object.hashCode).

Exercises — Part A: Coding

A1 — Set Intersection of Two Lists

```
// Return Set<T> intersection: elements present in both lists (one occurrence each)
public static <T> Set<T> intersection(List<T> list1, List<T> list2) {
    Set<T> set1 = new HashSet<>(list1);
    Set<T> result = new HashSet<>(list2);
    result.retainAll(set1); // keeps only elements present in both
    return result;
}
// Usage: intersection([1,2,3,3,4], [2,3,5]) -> {2,3}
```

A2 — Find Duplicate Elements in a List

```
// Return Set<String> of elements that appear more than once
public static Set<String> findDuplicates(List<String> list) {
    Set<String> seen = new HashSet<>();
    Set<String> duplicates = new HashSet<>();
    for (String s : list) {
        if (!seen.add(s)) { // add() returns false if already present
            duplicates.add(s);
        }
    }
    return duplicates;
}
// Usage: findDuplicates(["a","b","a","c","b","d"]) -> {"a","b"}
```

A3 — Case-Insensitive TreeSet

```
// TreeSet where all operations are case-insensitive
TreeSet<String> caseInsensitiveSet = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
caseInsensitiveSet.add("Apple");
caseInsensitiveSet.add("banana");
caseInsensitiveSet.add("apple"); // NOT added – case-insensitive duplicate
System.out.println(caseInsensitiveSet); // [Apple, banana]
System.out.println(caseInsensitiveSet.contains("APPLE")); // true
System.out.println(caseInsensitiveSet.floor("cherry")); // banana
// WARNING: comparator is inconsistent with equals -> Set contract technically violated
```

A4 — Set Difference as Immutable Set

```
// Return elements in setA but not in setB, as an immutable Set
public static <T> Set<T> difference(Set<T> setA, Set<T> setB) {
    return setA.stream()
        .filter(e -> !setB.contains(e))
        .collect(Collectors.toUnmodifiableSet());
}
// Mutable version:
public static <T> Set<T> differenceMutable(Set<T> setA, Set<T> setB) {
    Set<T> result = new HashSet<>(setA);
    result.removeAll(setB);
    return result;
}
```

A5 — Anagram Detection Using Set Operations

```
// Group a list of strings by anagram – strings that are anagrams of each other
public static Map<String, List<String>> groupAnagrams(List<String> words) {
    Map<String, List<String>> groups = new HashMap<>();
    for (String word : words) {
        // Sort characters to create canonical key
        char[] chars = word.toLowerCase().toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
    }
    return groups; // groups with size > 1 are anagram groups
}

// To detect if two strings are anagrams using Set operations:
// (Note: Sets lose frequency info, so this only works if no repeated chars)
public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    Set<Character> setA = new HashSet<>();
    for (char c : a.toCharArray()) setA.add(c);
    Set<Character> setB = new HashSet<>();
    for (char c : b.toCharArray()) setB.add(c);
    return setA.equals(setB); // Only reliable for strings with all unique chars
}
```

Exercises — Part B: Debug & Fix

B1 — TreeSet ClassCastException: Missing Comparable

```
class Product {
    String name; double price;
    public Product(String name, double price) {
        this.name = name; this.price = price;
    }
}
TreeSet<Product> products = new TreeSet<>();
products.add(new Product("Apple", 1.5));
products.add(new Product("Banana", 0.8)); // ClassCastException!
```

Fix: Product does not implement Comparable. TreeSet calls compareTo() when adding the second element. Fix: implement Comparable: `@Override public int compareTo(Product other) { return Double.compare(this.price, other.price); }` Or provide a Comparator at construction: `new TreeSet<>(Comparator.comparingDouble(p -> p.price));`

B2 — HashSet Not Detecting Duplicates: Missing hashCode

```
class Point {
    int x, y;
    Point(int x, int y) { this.x=x; this.y=y; }
    @Override public boolean equals(Object o) {
        Point p = (Point)o;
        return x==p.x && y==p.y;
    }
    // hashCode NOT overridden!
}
Set<Point> set = new HashSet<>();
set.add(new Point(1, 2));
set.add(new Point(1, 2)); // Should be duplicate!
System.out.println(set.size()); // 2 - WRONG
```

Fix: equals() is overridden but hashCode() is not. Different Point objects get different Object.hashCode() (identity-based), so they land in different buckets — HashSet never even calls equals(). Fix: `@Override public int hashCode() { return Objects.hash(x, y); }` Now both points get the same hashCode, land in the same bucket, and equals() correctly identifies them as duplicates.

B3 — ConcurrentModificationException During Set Iteration

```
Set<String> set = new HashSet<>(Arrays.asList("a", "b", "c", "d"));
for (String s : set) {
    if (s.equals("b")) {
        set.remove(s); // ConcurrentModificationException!
    }
}
```

Fix: Modifying the set directly during for-each iteration changes modCount, causing the iterator's next() to throw ConcurrentModificationException. Fix 1 — use Iterator.remove(): `Iterator it = set.iterator(); while (it.hasNext()) { String s = it.next(); if (s.equals("b")) it.remove(); }` Fix 2 — use removeIf() (Java 8+): `set.removeIf(s -> s.equals("b"));` Fix 3 — collect then remove: `set.remove("b");` (if removing known element, just call remove directly)

B4 — TreeSet contains() Returns False Due to Comparator>equals Inconsistency

```
TreeSet<String> ts = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
ts.add("Apple");
System.out.println(ts.contains("Apple")); // true
System.out.println(ts.contains("apple")); // true (via comparator)

// Bug scenario:
String key = new String("Apple"); // different object
HashSet<String> lookup = new HashSet<>(ts); // copies to HashSet
System.out.println(lookup.contains("apple")); // false! HashSet uses equals()!
```

Fix: TreeSet uses the Comparator for all operations (case-insensitive), but HashSet uses equals() (case-sensitive).

Copying from TreeSet to HashSet loses the case-insensitive semantic. TreeSet.contains("apple") returns true (comparator match). HashSet.contains("apple") returns false (equals() mismatch since element stored is "Apple"). Fix: be explicit about the comparison semantics. If case-insensitivity is needed, use a case-normalized key (e.g., store all lowercase) and a regular TreeSet/HashSet. Do not mix Set implementations with inconsistent comparison semantics.

Exercises — Part C: MCQ / Predict the Output

MC1. What is printed? `TreeSet ts = new TreeSet<>(Arrays.asList(5,2,8,1)); System.out.println(ts.floor(3));`

- A) 2
- B) 5
- C) null
- D) 3

Answer: A — `floor(3)` returns the greatest element ≤ 3 . The set contains $\{1,2,5,8\}$. Greatest element ≤ 3 is 2.

MC2. What is the output? `Set s = new LinkedHashSet<>(Arrays.asList("b","a","c","a","b")); System.out.println(s);`

- A) [a, b, c]
- B) [b, a, c]
- C) [b, a, c, a, b]
- D) [a, b, c, a, b]

Answer: B — `LinkedHashSet` preserves insertion order of FIRST occurrence. 'b' added first, then 'a', then 'c'. Duplicate 'a' and 'b' are rejected. Output: [b, a, c].

MC3. What happens? `EnumSet es = EnumSet.noneOf(Day.class); es.add(null);`

- A) null added successfully
- B) `NullPointerException`
- C) `IllegalArgumentException`
- D) `UnsupportedOperationException`

Answer: B — `EnumSet` does not allow null elements. `add(null)` throws `NullPointerException` immediately. This is different from `HashSet` which allows one null.

MC4. What is the output? `Set a = new HashSet<>(Arrays.asList(1,2,3)); Set b = new TreeSet<>(Arrays.asList(1,2,3)); System.out.println(a.equals(b) + " " + a.hashCode() == b.hashCode());`

- A) true true
- B) true false
- C) false true
- D) Compile error

Answer: A — `AbstractSet.equals()` compares by contents — same 3 elements so true. `AbstractSet.hashCode()` is defined as sum of element hashCodes — same elements, same sum. Note: `==` has lower precedence than `+`, so the expression is `(true + " " + (a.hashCode() == b.hashCode()))` printed as 'true true'.

MC5. What is printed? `TreeSet ts = new TreeSet<>(Arrays.asList(10,20,30)); SortedSet head = ts.headSet(20); ts.add(15); System.out.println(head);`

- A) [10]
- B) [10, 15]
- C) [10, 15, 20]
- D) [10, 20]

Answer: B — `headSet(20)` is a LIVE VIEW of all elements strictly less than 20. Adding 15 to the backing `TreeSet` automatically appears in the view since $15 < 20$. Output: [10, 15].

MC6. How many elements does this HashSet have? `Set s = new HashSet<>(); s.add("Java"); s.add("java"); s.add("JAVA"); System.out.println(s.size());`

- A) 1
- B) 2
- C) 3
- D) Depends on JVM

Answer: C — `String.equals()` is case-sensitive. "Java", "java", and "JAVA" are three distinct strings with different hashCodes. `HashSet` stores all three. Size is 3.

Quick Reference Cheat Sheet — Set Implementations

Topic	Key Points to Remember
HashSet internals	Backed by HashMap; elements = keys, PRESENT = dummy value; O(1) avg add/remove/contains
HashSet ordering	No order guarantee; iteration order may change after resize
HashSet null	Exactly one null allowed; null -> bucket 0 in backing HashMap
LinkedHashSet	Extends HashSet; backed by LinkedHashMap; insertion-order iteration via doubly-linked list through entries
LinkedHashSet null	Allows one null; same as HashSet; 25% more memory per entry (before/after pointers)
TreeSet internals	Backed by TreeMap (Red-Black tree); O(log n) guaranteed worst-case
TreeSet ordering	Natural order (Comparable) or Comparator; sorted ascending by default
TreeSet null	No null ever — compareTo(null) throws NPE; ClassCastException if no Comparable/Comparator
NavigableSet	floor/ceiling/lower/higher; headSet/tailSet/subSet with inclusive flags; pollFirst/pollLast
SortedSet views	Live views; modifications to view affect backing set; element range validated on add to view
EnumSet	Bit vector (one long for <=64 enum constants); O(1) all ops; ordinal = bit position
EnumSet factories	of(), allOf(), noneOf(), range(), copyOf(), complementOf() — never use new keyword
Set algebra	Union=addAll; Intersection=retainAll; Difference=removeAll; Disjoint=Collections.disjoint()
Immutable sets	Set.of() (Java 9+): no null, no dup; Set.copyOf() (Java 10+): snapshot; unmodifiableSet(): live wrapper
Thread safety	None of HashSet/LinkedHashSet/TreeSet/EnumSet are thread-safe
Concurrent options	ConcurrentSkipListSet (sorted,CAS); ConcurrentHashMap.newKeySet() (unordered); CopyOnWriteArraySet (tiny,read-heavy)
equals/hashCode	MUST override both; missing hashCode causes silent duplicates in HashSet
Comparator vs equals	TreeSet uses comparator ONLY (never equals); inconsistency causes silent duplicate loss or wrong contains()

End of Sub-Chapter 1.3 — Collections: Set Implementations

Next: [Sub-Chapter 1.4 — Map Implementations](#)